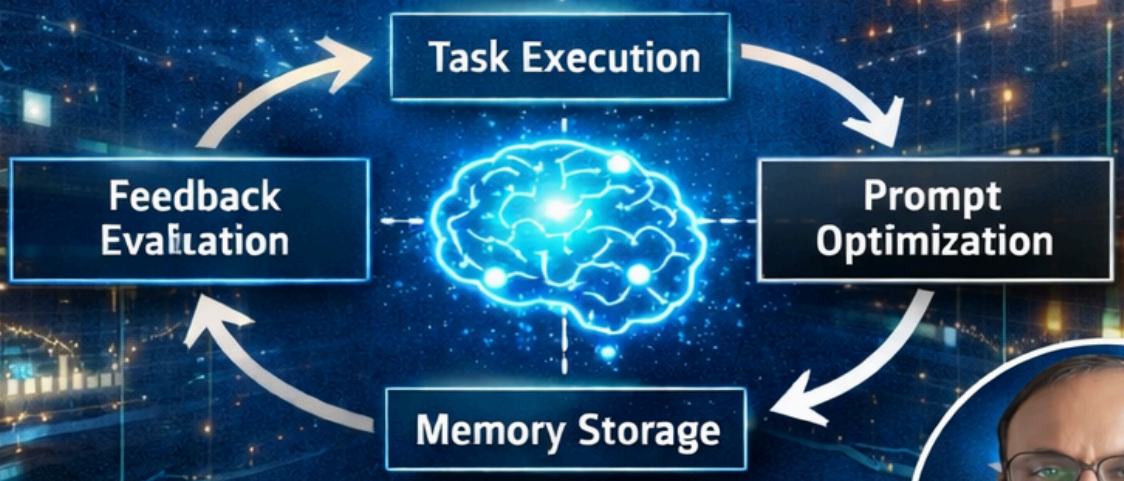
SELF-IMPROVING AI AGENT FRAMEWORK



Complete Developer's Guide
Implementation Manual

Selvagurunathan Ramalingam

Chapter 1: Introduction & Overview **3**
Problem It Solves 4
Real-World Applications 4
Who Should Use This? 5
What Makes It Unique? 5
Chapter 2: Architecture & Design **6**
System Architecture Overview 6
Chapter 3: Installation & Setup **13**

Chapter 1: Introduction & Overview

Git URL: https://github.com/rselvaguru/selfimprovingaiagent/

What is the Self-Improving AI Agent Framework?

The Self-Improving AI Agent Framework is a cutting-edge Python-based system that creates autonomous AI agents capable of evaluating their own performance and automatically improving their prompts through iterative feedback loops.

Key Innovation: Unlike traditional AI systems that use static prompts, this framework continuously learns and optimizes prompts based on evaluation feedback, resulting in progressively better responses.

Core Philosophy

The framework operates on three fundamental principles:

1. Autonomy: Agents operate independently without human intervention
2. Continuous Improvement: Each iteration refines the approach
3. Memory-Driven Learning: Previous experiences inform future decisions

Problem It Solves

Challenge: Traditional LLM systems produce responses of varying quality, with no built-in mechanism for improvement.
Solution: Our framework implements a feedback loop where:
- Responses are automatically evaluated
- Weaknesses are identified
- Prompts are optimized
- Results improve with each iteration

Real-World Applications

Code Generation
Initial: Basic function
After 2 iterations: Function with error handling
After 3 iterations: Documented function with type hints

Content Writing
Initial: First draft
After 2 iterations: Draft with improved structure
After 3 iterations: Polished, publication-ready content

Problem Solving
Initial: Basic solution
After 2 iterations: Verified solution
After 3 iterations: Optimized, tested solution

Who Should Use This?

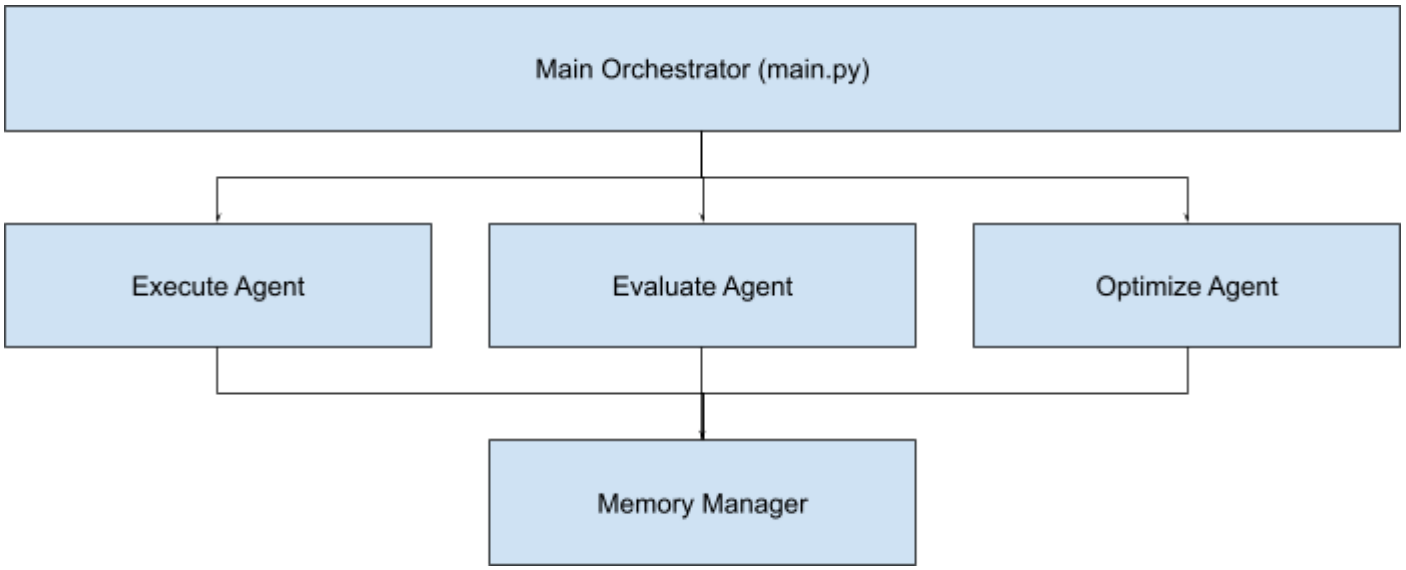
- Developers: Automate code generation with quality improvement
- Content Creators: Refine writing with AI assistance
- Researchers: Experiment with self-improving systems
- Businesses: Automate task execution with quality guarantees
- AI Enthusiasts: Learn advanced agent architecture

What Makes It Unique?

1. Self-Optimization: Prompts improve automatically
2. Semantic Memory: Remembers similar tasks for faster execution
3. Quality Scoring: Automatic evaluation (1-10 scale)
4. Modular Design: Easy to customize and extend
5. Multiple Interfaces: CLI, API, Python integration
6. Production Ready: Error handling, logging, configuration

Chapter 2: Architecture & Design

System Architecture Overview



The Learning Loop (Core Workflow)

Step 1: Task Input
User provides a task description:

"Write a Python function to calculate factorial"

Step 2: Prompt Construction
Base prompt + task is combined:

"You are an expert assistant. Complete the following task...
Task: Write a Python function to calculate factorial"

Step 3: Execution

ExecutorAgent sends to Ollama LLM:

- Ollama API receives prompt
- LLM generates response
- Response returned to framework

Step 4: Evaluation
EvaluatorAgent scores the response:

Score: 6/10
Feedback: "Missing error handling"
Weaknesses: ["No input validation"]
Suggestions: ["Add try-except block", "Check for negative numbers"]

Step 5: Decision Point
Framework checks score:

```
if score >= 8 or iterations == max_iterations:
   return result
else:
   continue to optimization
```

Step 6: Optimization
OptimizerAgent improves the prompt:

Old: "Write a Python function to calculate factorial"
New: "Write a robust Python function to calculate factorial
    with proper error handling for negative numbers

and input validation"

Step 7: Memory Storage
Everything is stored:

Task execution
Response generated
Evaluation score & feedback
Improved prompt version

Step 8: Iteration
Loop continues with improved prompt until score >= 8

Component Details

ExecutorAgent
Responsibility: Execute tasks using LLM
Key Methods:
- execute(task, prompt) - Run task with given prompt
- _call_ollama(prompt) - Send request to Ollama API

Temperature: 0.7 (balanced creativity)
Features:
- Ollama API integration
- Error handling
- Response parsing
- Logging

EvaluatorAgent
Responsibility: Score and provide feedback
Key Methods:
- execute(task, response) - Evaluate response
- _create_evaluation_prompt() - Generate eval prompt
- _parse_evaluation() - Parse LLM feedback

Temperature: 0.3 (consistent scoring)
Features:
- Quality scoring (1-10)
- Strength identification
- Weakness detection
- Suggestion generation

OptimizerAgent
Responsibility: Improve prompts
Key Methods:
- execute(task, feedback, suggestions) - Optimize prompt
- _create_optimization_prompt() - Generate optimization request
- _identify_changes() - Track modifications

Temperature: 0.5 (balanced improvement)
Features:
- Feedback incorporation
- Prompt enhancement
- Version tracking
- Change documentation

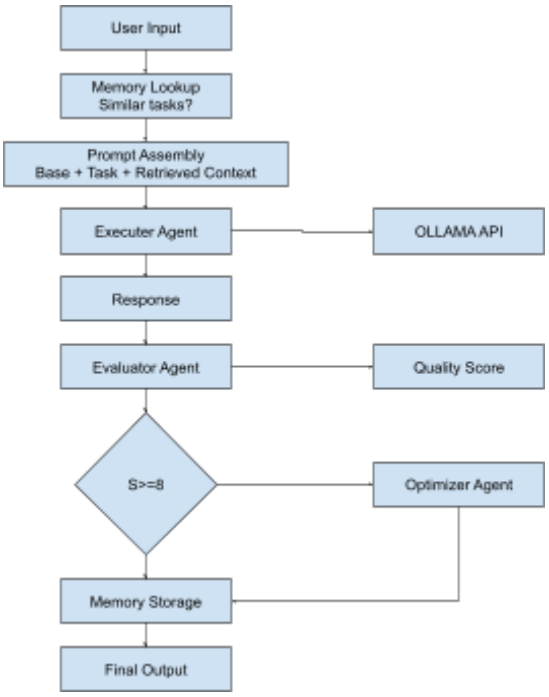
MemoryManager
Responsibility: Store and retrieve memories
Key Methods:

- store_task_execution() - Save task & response
- store_evaluation() - Save scores & feedback
- store_prompt_version() - Save optimized prompts
- retrieve_similar_tasks() - Find related tasks
- retrieve_best_prompt() - Get top performing prompt
- get_statistics() - Aggregate statistics

Features:

- ChromaDB vector storage
- Semantic search
- History tracking
- Analytics

Data Flow Diagram



Design Patterns Used

1. Abstract Factory Pattern

```python
class BaseAgent(ABC):
    @abstractmethod
    def execute(self, input_data):
        pass
class ExecutorAgent(BaseAgent):
    def execute(self, input_data):
        # Implementation
```

2. Strategy Pattern

Each agent implements different strategies for their role

3. Observer Pattern

The logging system observes and records all operations

4. Builder Pattern

Prompts are built incrementally from components

5. Repository Pattern

MemoryManager provides a repository interface for data

Scalability Considerations
Current Implementation:

- Single-threaded processing
- Single machine execution
- Local LLM inference
- In-memory agent instances

Scalability Options:
1. **Parallel Agents**: Run multiple agents simultaneously
2. **Distributed Memory**: Share memory across systems
3. **Multi-Model**: Use different models for different tasks
4. **Async Processing**: Non-blocking API calls
5. **Caching**: Cache similar task results

Chapter 3: Installation & Setup

System Requirements

Minimum:

- Python 3.8
- 4GB RAM
- 10GB storage
- macOS, Linux, or Windows with WSL2

Recommended:

- Python 3.10+
- 8GB RAM
- 20GB storage
- SSD storage
- GPU support (optional, for Ollama)

Step-by-Step Installation

Step 1: Install Prerequisites

Check Python:

```bash
python3 --version  # Should be 3.8+
```

Install Ollama:

- Visit https://ollama.ai
- Download installer for your OS
- Install and verify: ollama --version

Step 2: Setup Project
bash
Create project directory
mkdir -p ~/projects/ai-agent
cd ~/projects/ai-agent
Clone or download project files
(Download from repository or copy files)

Step 3: Create Virtual Environment
bash
Create environment
python3 -m venv venv

Activate environment
macOS/Linux:
source venv/bin/activate
Windows:
venv\Scripts\activate

Step 4: Install Dependencies
bash
Upgrade pip
pip install --upgrade pip
Install all dependencies
pip install -r requirements.txt

What Gets Installed:
requests (HTTP library)
fastapi (Web framework)
uvicorn (ASGI server)
pydantic (Data validation)
chromadb (Vector database)
langchain (LLM tools)
python-dotenv (Environment config)

Step 5: Download LLM Models
Terminal 1 (Keep running):
bash
```
ollama serve
```

Terminal 2 (Run once):
bash
Download a model
```
ollama pull llama2
```
Or choose alternatives:
```
ollama pull mistral     # Faster
ollama pull phi         # Smallest
ollama pull neural-chat  # Advanced
```

Verify Model:
bash
```
ollama list
```

Step 6: Verify Installation
bash
python setup.py

Expected Output:

Python 3.8+ OK
requests
fastapi
chromadb
All checks passed!

Configuration
config.py Settings
python
Ollama
OLLAMA_API_URL =
"http://localhost:11434/api/generate"
DEFAULT_MODEL = "llama2"
Temperature (0.0-1.0)
EXECUTOR_TEMPERATURE = 0.7 # More creative
EVALUATOR_TEMPERATURE = 0.3 # More
consistent
OPTIMIZER_TEMPERATURE = 0.5 # Balanced
Learning
MAX_ITERATIONS = 3
SCORE_THRESHOLD = 8
Memory

MEMORY_PERSIST_DIR = "./chroma_data"
API
API_HOST = "0.0.0.0"
API_PORT = 8000

Custom Configuration
Edit config.py for your needs:
python
Use a different model
DEFAULT_MODEL = "mistral"
More iterations for better quality
MAX_ITERATIONS = 5
Change threshold
SCORE_THRESHOLD = 9
Custom API port
API_PORT = 8080

Troubleshooting Installation
Issue: "Python 3.8+ required"
bash
Install via Homebrew (macOS)
brew install python@3.11
Or download from python.org

Issue: "Cannot connect to Ollama"
bash
Make sure Ollama is running
ollama serve
Check connectivity
curl http://localhost:11434/api/tags

Issue: "No models found"
bash
Pull a model
ollama pull llama2
Verify
ollama list

Issue: Virtual environment issues
bash
Recreate environment
rm -rf venv
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt